AUTOMATED TESTING AND CODE CONSISTENCY

6. Automated Testing

Automated testing was implemented through a continuous integration (CI) pipeline in GitHub Actions. The pipeline runs automatically on every push and pull request to the project's main branches (*Seb1* and *main*), without manual intervention. It consists of three parallel jobs, summarized in Table 7. A failure in any step marks the entire run as failed, which prevents defective changes from passing unnoticed.

Table 7

Continuous Integration Pipeline

Job	Automated Steps
Backend (PHP 8.2)	Validation of the Composer configuration; dependency installation; syntax checking of every PHP source file; code style verification with Laravel Pint; execution of all 72 unit and integration tests against a MySQL 8.4 database service
Client Portal	Dependency installation; security audit of production dependencies (fails on high or critical vulnerabilities); static analysis with ESLint; type-checked production build
Administrative Application	Dependency installation; security audit of production dependencies; production build

During the evaluation, the pipeline itself was found to be failing: both frontend applications depended on package versions with published high-severity vulnerabilities, so the security audit step failed. The affected packages (*zod* and *follow-redirects*) were updated to patched versions, after which all checks completed successfully. A PHP dependency audit also found a medium-severity e-mail header injection flaw in *symfony/mime* (CVE-2026-45070), which was resolved by updating to version 7.4.22. In the final verification on October 31, 2026, all 72 automated tests (432 assertions) passed, no code style violations or lint errors were found, no high-severity vulnerabilities remained, and both frontend applications built successfully.

Automated tests covered the backend, where the system's business rules, access control, and forecasting computations are implemented. The two React frontend applications were verified automatically through dependency audits, static analysis (client portal), and successful production builds, but they did not have automated user-interface tests. Their behavior was evaluated through system testing and user acceptance testing. In addition, the administrative application was exercised in a browser: the researchers signed

in as a clinic administrator and opened all thirteen of its pages, none of which failed to render.

All of the corrections described in this section were deployed to the live system. The hosting services build and publish the application automatically from the project repository, and the changes were confirmed on the deployed backend and administrative application by inspecting their responses after deployment.

7. Code Consistency

Code consistency was maintained through automated formatting and static analysis tools, so that code written by different members of the team follows a single convention. The standards and tools used are summarized in Table 8.

Table 8

Code Consistency Standards and Tools

Component	Standard / Tool	What It Enforces
Backend (PHP, Laravel)	Laravel Pint (Laravel preset, based on PSR-12)	Indentation, spacing, brace placement, quote style, import ordering, and removal of unused imports
Backend (all files)	EditorConfig	UTF-8 encoding, LF line endings, four-space indentation, a final newline, and removal of trailing whitespace
Client Portal (TypeScript, React)	ESLint with typescript-eslint and React Hooks rules	Unused variables and imports, empty blocks, correct use of React hooks, and use of *const* where values are not reassigned
Client Portal	TypeScript strict mode	Type checking during every build
Whole project	Git attributes	Binary handling of PDF documents to prevent line-ending corruption

When these tools were first applied, 255 of 381 backend PHP files did not conform to the project's formatting standard. The client portal's lint configuration was missing, so static analysis could not run. When it was restored, 239 lint errors were reported, mainly unused imports and variables, empty blocks, and code left unused after earlier changes. All backend files were reformatted with Laravel Pint, and the full test suite was re-run to confirm that the formatting changed no behavior. All 239 portal lint errors were resolved, including the removal of unused code. Both checks were then added to the CI pipeline, so any future change that violates the formatting standard or introduces a lint error causes the pipeline to fail.

The administrative application has no linter of its own, so it was checked once with a set of ESLint correctness rules (undefined variables, rules of React hooks, undefined JSX components, unreachable code, and similar). Across 112 files, this found eleven problems that would fail at run time, in four files. The calendar page called a styling helper (*clsx*) that was never imported, so the page failed to render. The species list called a function that no longer existed when a species was clicked. The main layout called two React hooks after an early return, which made the application fail instead of showing the maintenance notice when maintenance mode was switched on during a session. An inventory insights component was missing its React imports. All eleven were corrected, and the same check now reports no problems.

Two limitations remain. First, the client portal still reports 182 lint warnings, most of them explicit uses of the *any* type for API responses that are not yet fully typed. These are reported as warnings so that they remain visible and can be reduced over time. Second, the administrative application does not yet run a linter in the CI pipeline; the correctness check above was a one-time review. Adding static analysis and full type definitions to both frontend applications is recommended for future development.